First model submission (Model 1 – Random Forest non-linear regression)

Step 1 – Import Libraries

In [1]:
```
# Step 1: Imports for Model 1
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_log_error
from sklearn.preprocessing import LabelEncoder
```

Step 2 – Load Kaggle datasets

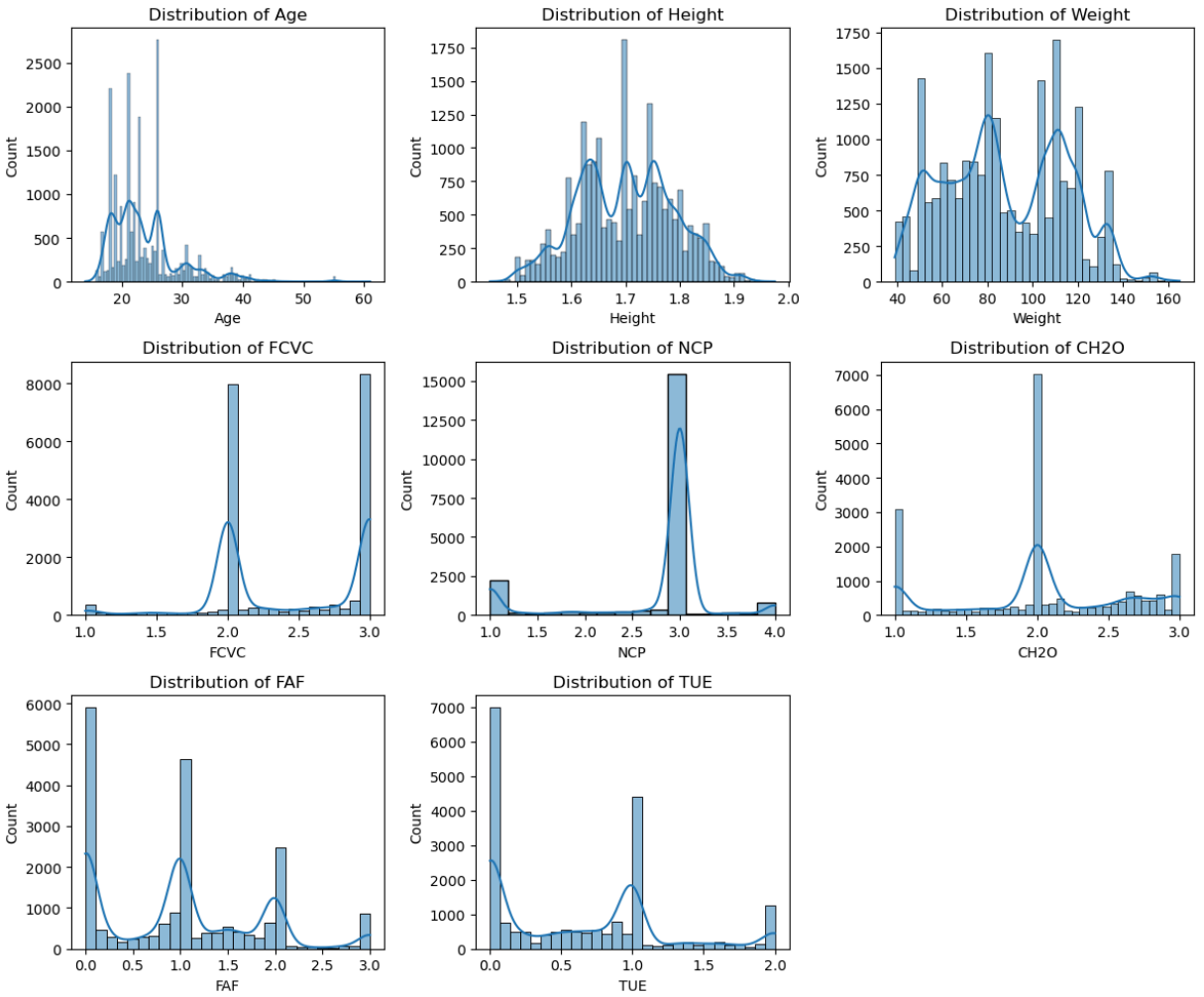
In [2]:
```
# Step 2: Load Kaggle data
train = pd.read_csv("train.csv")
test = pd.read_csv("test.csv")
sample_submission = pd.read_csv("sample_submission.csv")

train.head()
```

Out[2]:

	id	Gender	Age	Height	Weight	family_history_with_overweight	FAVC	
0	0	Male	24.443011	1.699998	81.669950	yes	yes	2.00
1	1	Female	18.000000	1.560000	57.000000	yes	yes	2.00
2	2	Female	18.000000	1.711460	50.165754	yes	yes	1.88
3	3	Female	20.952737	1.710730	131.274851	yes	yes	3.00
4	4	Male	31.641081	1.914186	93.798055	yes	yes	2.67

In [3]:
```
# 3. EDA + Figures Distribution Plots / Warning

import warnings

warnings.filterwarnings("ignore", category=FutureWarning)

import os
import matplotlib.pyplot as plt
import seaborn as sns

# 3. EDA + Figures Distribution Plots
numeric_cols = ["Age", "Height", "Weight", "FCVC", "NCP", "CH2O", "FAF", "TUE"]

# Automatically creates the "figures" folder if it doesn't exist yet
os.makedirs("NU/DDS8555/Build_Non-Linear_Models_Part 2/figures", exist_ok=True)


plt.figure(figsize=(12, 10))
```

```
for i, col in enumerate(numeric_cols):
    plt.subplot(3, 3, i + 1)
    sns.histplot(train[col], kde=True)
    plt.title(f"Distribution of {col}")

plt.tight_layout()
plt.savefig("NU/DDS8555/Build_Non-Linear_Models_Part 2/figures/model1_eda_distribut
plt.show()
```



Step 4 — Basic EDA + Missing Data Check

In [4]:
```
# Step 4: Quick EDA
print(train.info())
print(train.describe())

print("Missing values in train:")
print(train.isna().sum())

print("Missing values in test:")
print(test.isna().sum())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20758 entries, 0 to 20757
Data columns (total 18 columns):
 #   Column                          Non-Null Count  Dtype  
---  ------                          --------------  -----  
 0   id                              20758 non-null  int64  
 1   Gender                          20758 non-null  object 
 2   Age                             20758 non-null  float64
 3   Height                          20758 non-null  float64
 4   Weight                          20758 non-null  float64
 5   family_history_with_overweight  20758 non-null  object 
 6   FAVC                            20758 non-null  object 
 7   FCVC                            20758 non-null  float64
 8   NCP                             20758 non-null  float64
 9   CAEC                            20758 non-null  object 
 10  SMOKE                           20758 non-null  object 
 11  CH2O                            20758 non-null  float64
 12  SCC                             20758 non-null  object 
 13  FAF                             20758 non-null  float64
 14  TUE                             20758 non-null  float64
 15  CALC                            20758 non-null  object 
 16  MTRANS                          20758 non-null  object 
 17  NObeyesdad                      20758 non-null  object 
dtypes: float64(8), int64(1), object(9)
memory usage: 2.9+ MB
None
                id           Age        Height        Weight          FCVC  \
count  20758.00000  20758.000000  20758.000000  20758.000000  20758.000000   
mean   10378.50000     23.841804      1.700245     87.887768      2.445908   
std     5992.46278      5.688072      0.087312     26.379443      0.533218   
min        0.00000     14.000000      1.450000     39.000000      1.000000   
25%     5189.25000     20.000000      1.631856     66.000000      2.000000   
50%    10378.50000     22.815416      1.700000     84.064875      2.393837   
75%    15567.75000     26.000000      1.762887    111.600553      3.000000   
max    20757.00000     61.000000      1.975663    165.057269      3.000000   

                NCP          CH2O           FAF           TUE  
count  20758.000000  20758.000000  20758.000000  20758.000000  
mean       2.761332      2.029418      0.981747      0.616756  
std        0.705375      0.608467      0.838302      0.602113  
min        1.000000      1.000000      0.000000      0.000000  
25%        3.000000      1.792022      0.008013      0.000000  
50%        3.000000      2.000000      1.000000      0.573887  
75%        3.000000      2.549617      1.587406      1.000000  
max        4.000000      3.000000      3.000000      2.000000  
Missing values in train:
id                                0
Gender                            0
Age                               0
Height                            0
Weight                            0
family_history_with_overweight    0
FAVC                              0
FCVC                              0
NCP                               0
CAEC                              0
```

```
SMOKE                             0
CH2O                              0
SCC                               0
FAF                               0
TUE                               0
CALC                              0
MTRANS                            0
NObeyesdad                        0
dtype: int64
Missing values in test:
id                                0
Gender                            0
Age                               0
Height                            0
Weight                            0
family_history_with_overweight    0
FAVC                              0
FCVC                              0
NCP                               0
CAEC                              0
SMOKE                             0
CH2O                              0
SCC                               0
FAF                               0
TUE                               0
CALC                              0
MTRANS                            0
dtype: int64
```

Step 5 — Encode Categorical Variables

In [5]:
```python
#5 fit the encoder on the combined unique values of both the train and test sets.
# This ensures the encoder learns every possible category beforehand

import pandas as pd
from sklearn.preprocessing import LabelEncoder

# Step 5: Encode categorical features BEFORE splitting
cat_cols = train.select_dtypes(include=["object"]).columns

label_encoders = {}
for col in cat_cols:
    le = LabelEncoder()

    if col in test.columns:
        # FIX: Combine unique values from both train and test so 'Always' is recogn
        combined_categories = pd.concat([train[col], test[col]]).dropna().unique()
        le.fit(combined_categories)

        # Transform both datasets safely
        train[col] = le.transform(train[col])
        test[col] = le.transform(test[col])
    else:
        # For columns ONLY in train (like your target variable 'NObeyesdad')
        train[col] = le.fit_transform(train[col])
```

```
    label_encoders[col] = le
```

Step 6 — Create Features, Target, and Validation Split

In [6]:
```
# Step 6: Encode categorical features BEFORE splitting
cat_cols = train.select_dtypes(include=["object"]).columns

label_encoders = {}
for col in cat_cols:
    le = LabelEncoder()
    train[col] = le.fit_transform(train[col])
    test[col] = le.transform(test[col])
    label_encoders[col] = le
```

Step 7 — Fit Non-Linear Model 1 (Random Forest)

In [7]:
```
# Step 7: Define features and target

target_col = "NObeyesdad"

X = train.drop(columns=[target_col])
y = train[target_col]

X_test = test.copy()

# Train/validation split
X_train, X_valid, y_train, y_valid = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

In [8]:
```
# Step 8: Random Forest model
rf_model = RandomForestRegressor(
    n_estimators=400,
    max_depth=None,
    min_samples_leaf=1,
    random_state=42,
    n_jobs=-1
)

rf_model.fit(X_train, y_train)

# Validation predictions + RMSLE
y_valid_pred = rf_model.predict(X_valid)
rmsle_rf = np.sqrt(mean_squared_log_error(y_valid, y_valid_pred))
print("Model 1 - Random Forest RMSLE (validation):", rmsle_rf)
```

```
Model 1 - Random Forest RMSLE (validation): 0.2289190454631347
```

Step 9 — Retrain Random Forest on Full Training Data

In [9]:
```
# Step 9: Retrain on full training data
rf_model_full = RandomForestRegressor(
    n_estimators=400,
    max_depth=None,
```

```
    min_samples_leaf=1,
    random_state=42,
    n_jobs=-1
)

rf_model_full.fit(X, y)

# Predict on Kaggle test set
test_pred_rf = rf_model_full.predict(X_test)
```

Step 10 — Build First Submission File

In [10]:
```
# Step 10: Create first submission DataFrame
submission_rf = sample_submission.copy()
submission_rf["Rings"] = test_pred_rf

# Save CSV for Kaggle
submission_rf.to_csv("NU/DDS8555/Build_Non-Linear_Models_Part 2/submission/submissi

submission_rf.head()
```

Out[10]:

	id	NObeyesdad	Rings
0	20758	Normal_Weight	3.0000
1	20759	Normal_Weight	4.1600
2	20760	Normal_Weight	4.0000
3	20761	Normal_Weight	2.2775
4	20762	Normal_Weight	4.0000

In [ ]: